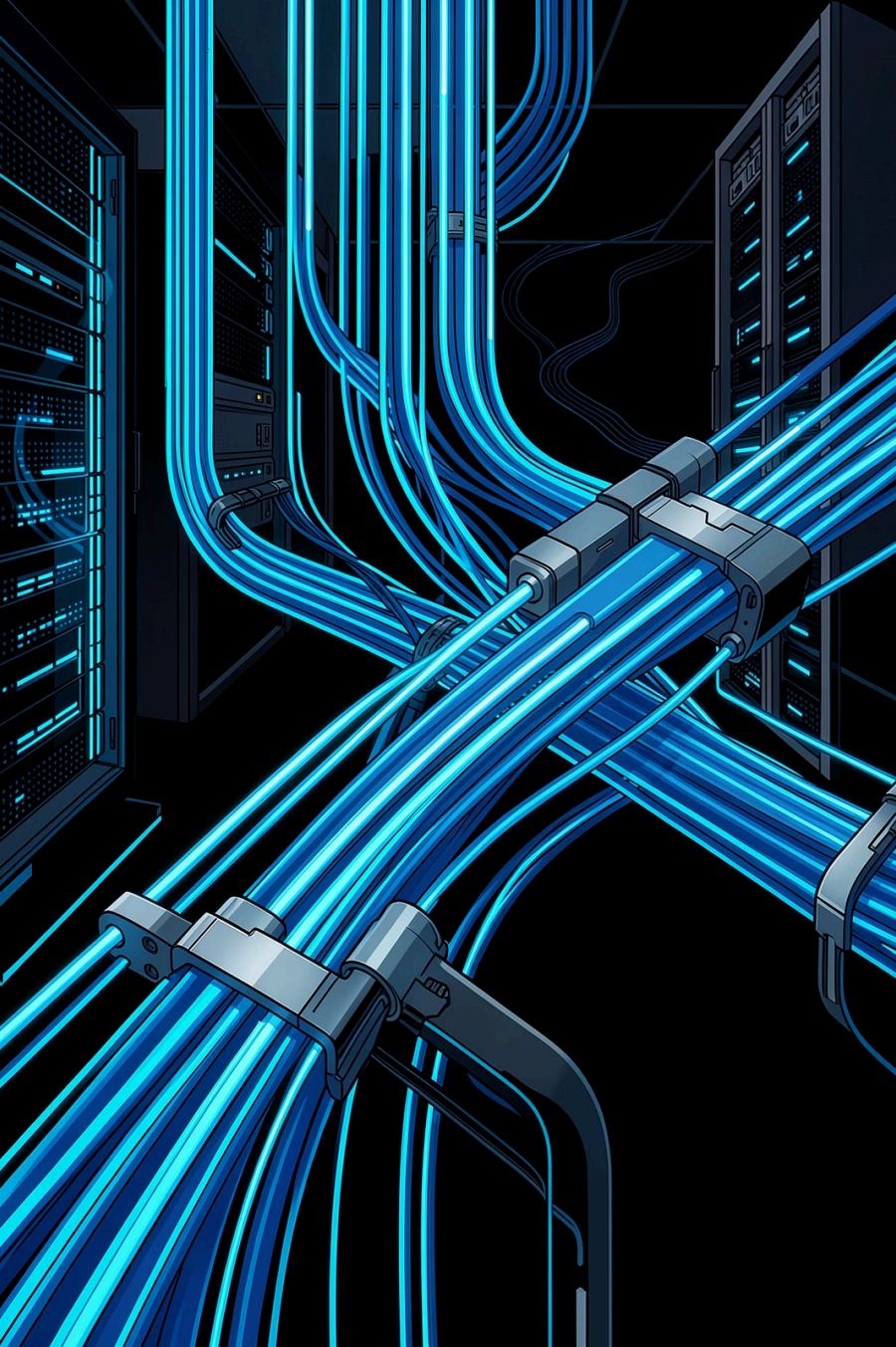




Reproducing BBR: A Controlled Study of Bottleneck-Based Congestion Control

CS 552 — Computer Networks · Spring 2026 · Rutgers University

Mihir Kulkarni · Chaitanya Ranaware · Piyoosha Gadi · Prof. Minsung Kim

- We empirically reproduced Google's BBR algorithm against CUBIC and RENO on a controlled Linux testbed. Three experiments, one finding: BBR trades a small throughput gap for much lower latency.

The Problem: Loss-Based Congestion Control

For over 30 years, TCP congestion control has relied on a single signal: **packet loss**. Algorithms like RENO and CUBIC accelerate until packets drop, then back off. This worked when router buffers were small — but modern hardware ships with megabytes of buffering, and the old model breaks down.

Buffer Bloat

A loss-based sender fills the router buffer *before* receiving a loss signal. Throughput looks fine, but queuing delay balloons — latency spikes even while the link is fully utilized.

The Core Question

Can we perform congestion control **without using loss as the signal**?
BBR's answer: model the physical properties of the path itself — bandwidth and propagation delay — and operate at the optimal point.

BBR's Idea: Model the Path

BBR estimates two physical quantities and uses their product — the **Bandwidth-Delay Product (BDP)** — as the target for in-flight data. This keeps the link full without queuing packets.



BtlBw — Bottleneck Bandwidth

The maximum delivery rate observed on the path. BBR paces its sending rate at this value to fully utilize the link.



RTprop — Propagation Delay

The minimum RTT observed, representing the path's physical propagation delay with no queuing. BBR uses this as its latency floor.



$$\text{BDP} = \text{BtlBw} \times \text{RTprop}$$

The optimal in-flight data volume. BBR keeps congestion window near BDP — full link, empty queue.



Four-Phase Cycle

STARTUP, DRAIN, PROBE_BW, and PROBE_RTT cycle continuously to keep estimates fresh and adapt to path changes.

 Linux kernel since 4.9 (2016). Deployed across Google's WAN, YouTube, and QUIC.

What We Built: The Testbed

A reproducible, low-overhead environment using Linux network namespaces and traffic control — no VMs, no virtualization noise.

Topology

Two network namespaces on one CachyOS host (kernel 6.19), connected by a **veth pair**:

- **ns1 (server)** — 10.0.0.1
- **ns2 (client)** — 10.0.0.2
- **tc tbf + netem** on the veth pair emulates the bottleneck

TSO/GSO/GRO disabled so packet-count queue limits reflect real packets.

Traffic Control Stack

tbf (root qdisc)

Enforces the rate limit on the bottleneck link.

netem (child qdisc)

Adds one-way delay and a packet-count queue limit.
Applied symmetrically so $RTT = 2 \times \text{one-way delay}$.

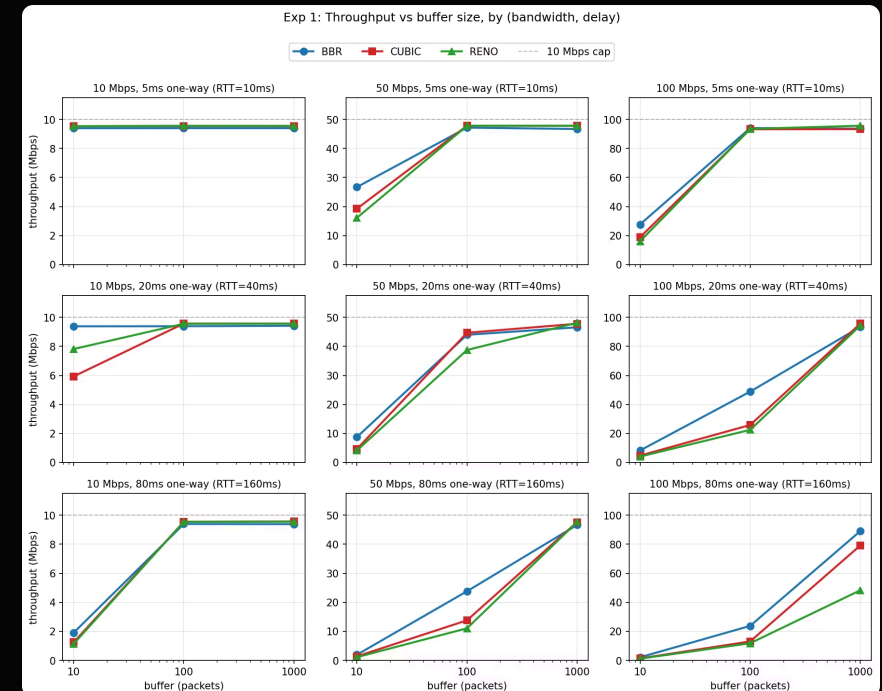
Traffic generated with `iperf3 -C <bbr|cubic|reno>`, JSON output at 100 ms intervals.

Throughput Sweep: 81 Configurations

We swept 3 CCs × 3 bandwidths × 3 delays × 3 buffer sizes — **81 runs at 30 seconds each** — measuring how close each algorithm gets to the bottleneck cap.

Selected Results at 100 Mbps Bottleneck (% of link capacity)

One-Way Delay / Buffer	BBR	CUBIC	RENO
5 ms (10 ms RTT) — any buffer	94%	93%	96%
20 ms (40 ms RTT) — 100 pkt	49%	26%	23%
80 ms (160 ms RTT) — 100 pkt	24%	13%	12%
80 ms (160 ms RTT) — 1000 pkt	89%	79%	48%



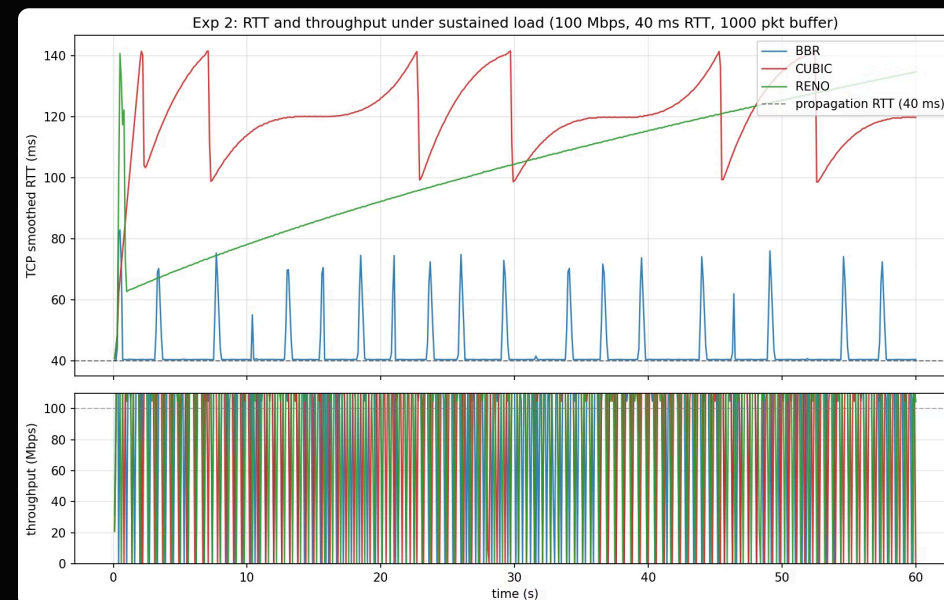
Throughput plot. Measured throughput across the 81 experiment configurations, showing how BBR, CUBIC, and RENO respond to bandwidth, delay, and buffer size.

- ✓ **Key Finding:** At shallow-to-moderate buffers on high-RTT links, all three CCs drop below capacity — but BBR consistently leads. On deep buffers, BBR matches CUBIC's throughput at a fraction of the latency cost (see Experiment 2). RENO never catches up at high RTT.

Buffer Bloat: Same Throughput, 3× the Latency

A single 60-second flow per CC at 100 Mbps, 40 ms base RTT, and a 1000-packet buffer ($\sim 3\times$ BDP). All three algorithms reached similar throughput — but latency tells a different story.

CC	Throughput	Mean RTT	RTT Inflation	Retransmits
BBR	92.9 Mbps	43.0 ms	1.07×	0
CUBIC	95.3 Mbps	120.3 ms	2.95×	93
RENO	95.3 Mbps	102.9 ms	2.53×	860



BBR's Advantage

BBR keeps latency at the propagation floor (~ 42 ms vs. 40 ms base). It never overfills the queue, resulting in **zero retransmits** in steady state.

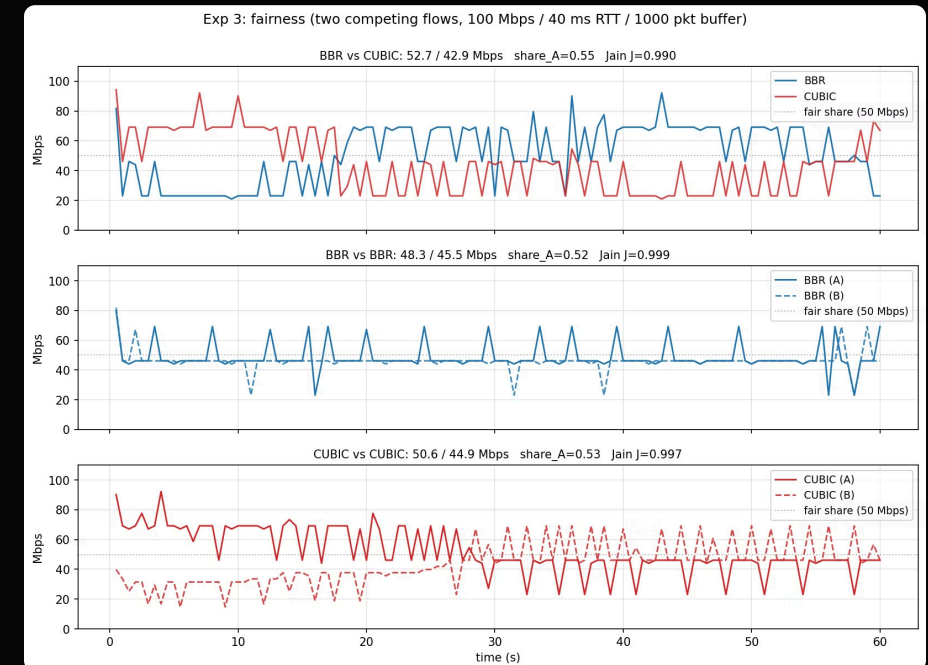
Loss-Based Penalty

CUBIC and RENO inflate RTT by $\sim 3\times$ while achieving nearly identical throughput. RENO's 860 retransmits reflect aggressive queue overfilling and loss recovery cycles.

Fairness: Competing Flows on the Same Bottleneck

Two iperf3 flows competing on a 100 Mbps / 40 ms / 1000-pkt bottleneck. We measured steady-state share split and **Jain's fairness index** (1.0 = perfect fairness).

Pairing	Share Split	Jain's Index	Interpretation
BBR vs. CUBIC	0.55 / 0.45	0.990	Near-perfect fairness
BBR vs. BBR	0.52 / 0.48	0.999	Perfect intra-protocol fairness
CUBIC vs. CUBIC	0.53 / 0.47	0.997	Fair, as expected



BBR vs. BBR Baseline

Two BBR flows split bandwidth nearly perfectly (Jain = 0.999), the intra-protocol baseline.

BBR vs. CUBIC: Fairer Than Expected

BBR took only a 55/45 share against CUBIC — far less unfair than the 2016 paper reported. Likely cause: kernel-side fairness patches accumulated in Linux BBR since publication.

BBRv2 Context

BBRv2 was developed to further tighten BBR-vs-CUBIC fairness in deep-buffer regimes, building on the original findings.

How Our Results Compare to the Paper

✓ Reproduced Cleanly

- BBR consistently outperforms CUBIC and RENO across all delay/buffer conditions
- CUBIC and RENO under-utilize on shallow buffers + high RTT
- BBR keeps RTT at the propagation floor; loss-based CCs inflate RTT 2.5–3x
- BBR has zero retransmits in steady state

⚠ Milder Than the Paper

- Likely cause: Linux BBR fairness patches accumulated since publication; our 3x-BDP buffer is also milder than the paper's deeper buffers
- Fairness Holds in Modern Linux: BBR vs CUBIC settles at a 55/45 split (Jain 0.990) in our runs — much fairer than the 2016 paper. The dramatic unfairness from BBRv1 has been substantially patched in modern kernels; BBRv2 tightens this further in deep-buffer regimes.

🔬 Scope Limitations

- No lossy-link experiments (random loss via netem)
- Multi-flow scaling limited to 2 flows
- BBRv2 not evaluated

Conclusions

1 Loss-Based CC Is a Latency Tax

BBR removes most of it by operating at the Kleinrock optimal point — full link, empty queue — without waiting for loss signals.

3 Fairness Holds in Modern Linux

BBR vs CUBIC settled at 55/45 (Jain 0.990) in our runs - the dramatic unfairness from the paper has been substantially patched. BBRv2 tightens this further in deep-buffer regimes

2 BBR's Win Is Context-Dependent

Largest on high-RTT shallow-buffer paths and deep-buffer paths where buffer bloat dominates. On short-RTT paths with small buffers, all CCs perform similarly.

4 A Laptop Testbed Suffices

A controlled `netns` + `tc` setup reproduces the paper's qualitative claims — accessible, reproducible, and low-overhead.

Throughput isn't the whole story. Latency under load is what BBR fixes.